

Fundamentos de la programación 2

Grado en Desarrollo de Videojuegos

Examen final. Convocatoria extraordinaria, junio 2025

Indicaciones generales:

- Se entregarán los archivos `Program.cs`, `Tablero.cs` y `Lista.cs`, con las clases correspondientes. Se proporciona una **plantilla** con el esquema del programa y algunos métodos ya implementados.
- En todos los archivos las líneas 1 y 2 serán comentarios de la forma:

```
// Nombre Apellido1 Apellido2  
// Laboratorio, puesto
```
- **Lee atentamente el enunciado** e implementa el programa tal como se pide, con las clases, tipos de datos, métodos, parámetros y requisitos que se especifican. No puede modificarse la representación propuesta, ni alterar los parámetros de los métodos pedidos. No se especifica el modo de paso de los parámetros en los métodos (`out`, `ref`, ...), que debe determinar el alumno.

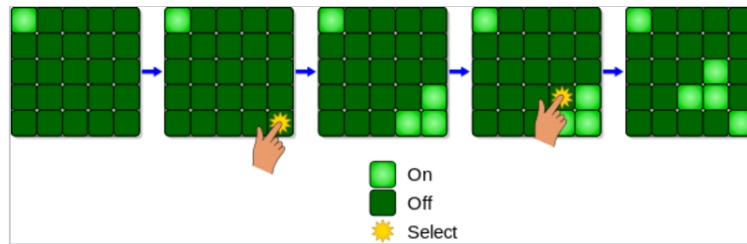
Pueden implementarse todos los métodos adicionales que se consideren oportunos, especificando claramente su cometido, parámetros, etc.

- **El programa debe compilar y funcionar correctamente, y estar bien estructurado y comentado.** Se valorarán la claridad, la concisión y la eficiencia.
 - La **entrega**: se realizará a través del servidor FTP de los laboratorios.
Importante: verificar que los archivos entregados son los requeridos.
-

Vamos a implementar el juego *Lights Out* (Tiger Electronics, 1995).

Consiste en una cuadrícula de luces de tamaño $FILS \times COLS$ (originalmente, 5×5) con algunas de ellas encendidas y otras apagadas. Al pulsar cualquiera de las luces, esta cambia su estado y el de las adyacentes (arriba, abajo, izquierda y derecha). El objetivo del juego es apagar todas las luces. A

continuación se muestra el aspecto del juego y el resultado de pulsar dos luces (Wikipedia, Lights Out (game)):



La clase `Tablero` implementará la funcionalidad principal para gestionar el tablero de juego. En el archivo `Tablero.cs` se proporciona una plantilla con la representación que utilizaremos:

```
class Tablero{
    int FILS, COLS;      // numero de filas y columnas del tablero
    bool [,] mat;        // matriz de casillas del tablero
    struct Posicion { public int fil, col; }    // tipo para coordenadas 2D
    Posicion cursor;     // coordenadas de la posicion del cursor
    ...
}
```

Los atributos `FILS` y `COLS` determinan el tamaño del tablero de juego. El *estado* del juego vendrá representado por la matriz de booleanos `mat` (`false` denota luz apagada y `true` encendida) y por la posición del cursor `cursor`. Implementaremos una constructora y el dibujo del tablero:

- **[0.5 pt]** `public Tablero(string [] lineas):` inicializa el tablero con la cuadrícula que viene representada en `lineas`, que será un array con un strings por cada fila (asumimos que codifica una cuadrícula correcta en dimensiones y contenido). Inicializa las dimensiones `FILS`, `COLS` y sitúa el cursor en (0,0). En los strings, el caracter `'*` representa luz encendida y `'.'` luz apagada. Desde el método `Main` podemos construir un tablero del siguiente modo:

```
string [] lineas = {
    "***.", // fila 0
    ...
}
```

```

    "...*", // fila 1

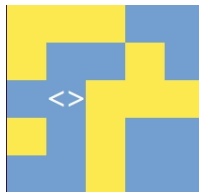
    "...**", // ...

    "...*",
    "...*" };

Tablero t = new Tablero(lineas); //llamada a la constructora

```

- **[0.5 pt]** public Dibuja(): muestra el tablero en pantalla. Para mejorar el aspecto visual, **cada casilla del tablero ocupará dos caracteres en pantalla**, como hemos hecho en otras ocasiones. Mostraremos las luces apagadas en azul (Console.BackgroundColor = ConsoleColor.Blue) y las encendidas en amarillo (idem con DarkYellow). En la posición del cursor escribiremos "<>". Para el tablero de ejemplo del método anterior, con el cursor en (2, 1), tendremos:



- **[1 pt]** private void ClickCasilla(): modifica la matriz mat pulsando la casilla determinada por el cursor, es decir, conmuta el estado de dicha casilla y sus adyacentes.
- **[0.5 pt]** public void FinJuego(bool fin): devuelve true en el parámetro fin si todas las luces están apagadas, false en caso contrario.

En la clase MainClass se proporciona ya implementado el método LeeInput, que recoge el input de usuario y devuelve un carácter que codifica la acción a realizar: mover el cursor, pulsar casilla, terminar juego (y otros que utilizaremos más adelante). Para procesar ese input implementaremos el método ProcesaInput en la clase Tablero:

- **[0.5 pt]** `public void ProcesaInput(char d):` modifica el estado del tablero de acuerdo al argumento `d`. En esta primera versión responderá a los siguientes caracteres:
 - `'l', 'r', 'u', 'd'`: modifican la posición del cursor a la izquierda, derecha, arriba y abajo, respectivamente;
 - `'c'`: representa la pulsación sobre la casilla donde está el *cursor* (llama a `ClickCasilla`). Nótese que `LeeInput` devuelve `'c'` cuando el usuario pulsa *Espacio*).
 - `'q'`: termina el juego (`LeeInput` devuelve este input al pulsar *Escape*).

En la clase `MainClass` implementaremos el método `Main` para tener una primera versión del juego:

- **[0.5 pt]** `public static void Main()`: crea un nuevo `Tablero` con la cuadrícula de ejemplo vista en la constructora e implementa el bucle principal que realiza en cada iteración (en este orden): lee el input de usuario, lo procesa y dibuja el estado. Terminará cuando todas las luces estén apagadas o el usuario decida abandonar.

A continuación implementaremos algunas extensiones. En primer lugar, implementaremos otra constructora (sobrecarga) que genere tableros aleatorios de tamaño dado `fils × cols`. Para garantizar que el tablero generado tiene solución utilizaremos el siguiente algoritmo:

- partimos de un tablero con todas las luces apagadas,
- a continuación pulsamos `n` casillas seleccionadas aleatoriamente.

Es fácil razonar que una solución sería pulsar esas mismas `n` casillas en orden inverso. Así pues, implementaremos este algoritmo en una nueva constructora para la clase `Tablero`:

- **[1 pt]** `public Tablero(int fils, int cols, int n)`: crea un tablero de tamaño `fils × cols` con todas las luces apagadas. Después genera una configuración inicial pulsando `n` casillas (`ClickCasilla`) seleccionadas aleatoriamente tal como acabamos de describir.

A continuación implementaremos la opción de deshacer jugadas. Nótese que para deshacer una jugada basta volver a pulsar la misma casilla. Para poder deshacer más jugadas tendremos que *memorizar* las posiciones de las casillas pulsadas, para lo que utilizaremos una lista. Se proporciona ya implementada la clase `Lista` para listas de enteros, pero como necesitamos guardar pares

(x, y) hacemos lo siguiente: guardamos el valor $k = 10 * x + y$; después recuperaremos el par original utilizando división entera y resto. En la clase Tablero añadimos como atributo la lista MemoJugadas (que debe inicializarse en la constructora):

```
class Tablero{  
    ...  
    // lista de posiciones para guardar jugadas y poder deshacerlas  
    Lista MemoJugadas;  
}
```

Ahora cada vez que se pulsa una casilla se se procede como antes, pero además se guarda su posición al principio de la lista MemoJugadas. Para deshacer una jugada se toma la primera posición de esta lista, se pulsa en la casilla correspondiente y se elimina dicha posición de la lista; para deshacer otra se repite el proceso, tantas veces como se desee o hasta que no queden jugadas por deshacer. Por ejemplo, si pulsamos las casillas $(4, 2)$, $(1, 4)$, $(0, 3)$ en este orden, tendremos la lista (codificando estos pares se ha explicado):

$$\boxed{3} \longrightarrow \boxed{14} \longrightarrow \boxed{42}$$

Para deshacer la última jugada tomamos el primer elemento 3 (que corresponde a la casilla $(0, 3)$), la pulsamos (ClickCasilla) y la eliminaríamos de la lista. Para deshacer otra jugada, haríamos lo mismo, ahora con $(1, 4)$, etc. Para implementar este mecanismo se piden dos nuevos métodos privados en la clase Tablero:

- **[2 pt]** void ClickCasillaMemo() y void DeshacerJugada().

Ahora, en ProcesaInput se invocará a ClickCasillaMemo en vez de a ClickCasilla cuando se pulse una casilla y con la tecla 'z' se deshará la última jugada.

Otra opción útil es poder guardar y recuperar partida utilizando archivos. Para ello guardaremos la disposición del tablero en una única cadena de texto,

donde '*' representa luz encendida, '.' luz apagada y '|' marca el final de cada fila. Por ejemplo, para el tablero de ejemplo de la constructora tendremos la cadena:

```
***..|*...|...***|*.*..|...*..
```

En el método `ProcesaInput` añadiremos las opciones 's' para salvar partida y 'o' para restaurar una partida salvada, que invocarán a los siguientes métodos privados en la clase `Tablero`:

- **[0.5 pt]** `void Guardar()`: genera un archivo de texto de nombre `partida.txt` con el contenido del tablero en el formato explicado;
- **[1.5 pt]** `void Restaurar()`: recupera una partida guardada del archivo `partida.txt`. Comprobará que dicho archivo contiene un tablero válido (cadena de texto con el formato explicado y con dimensiones correctas) y utilizará la (primera) constructora para crear el tablero. En el caso de que el archivo no exista o el formato sea incorrecto se **lanzaré una excepción**, que se capturara en el lugar correspondiente, mostrando un mensaje de error.

Una última opción interesante será la de restaurar el estado inicial de una partida, tras una serie de jugadas. Esto puede realizarse deshaciendo todas las jugadas realizadas hasta el momento. Para ello añadiremos la opción 'i' en `ProcesaInput` y a la clase `Tablero` el siguiente método recursivo:

- **[1 pt]** `void Reinicia()`: utilizando la lista `MemoJugadas`, deshace **recursivamente** todas las jugadas realizadas hasta el momento. **Nota:** no se valorán las soluciones iterativas.

¿Podría hacerse este método recursivo final? Justifica tu respuesta.

Por último, se pide **[0.5 pt]** integrar todas estas opciones en el juego para que queden accesibles a través de la interfaz de usuario.